Universidad del Rosario | Escuela de Ciencias e Ingeniería | MACC

Programación orientada a Objetos: Fundamentos

Clases, Objetos, Encapsulamiento y Herencia

Objetivos

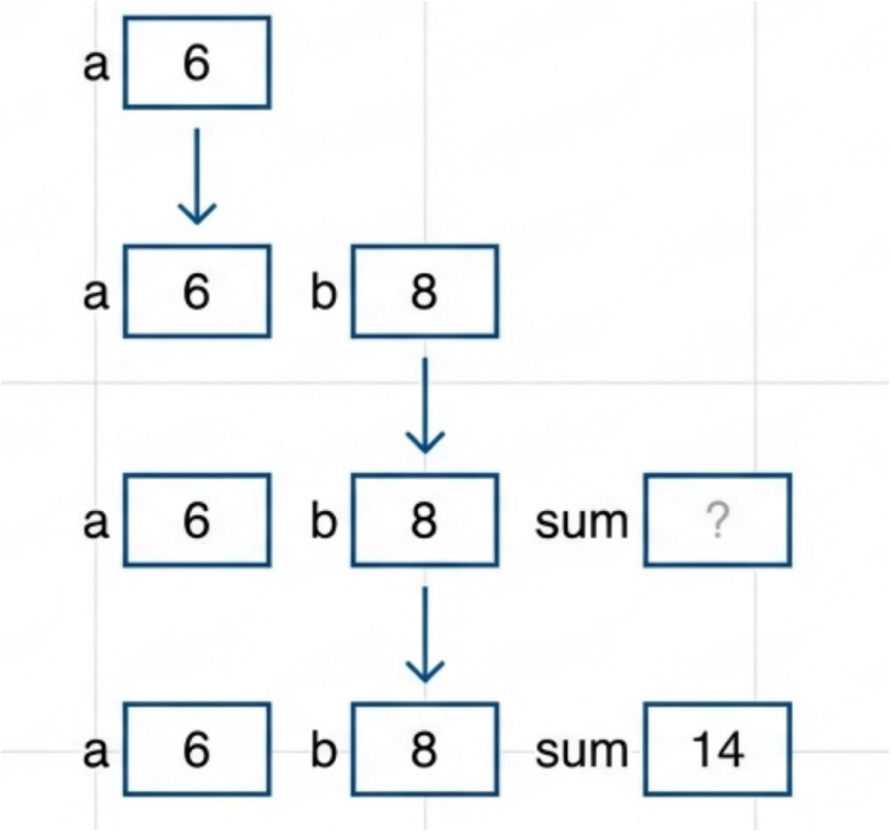
- Comprender la POO como un modelo para gestionar la complejidad mediante abstracción
- Diferenciar entre el paradigma procedural y el orientado a objetos
- Entender los cuatro pilares fundamentales: Encapsulamiento, Abstracción, Herencia y Polimorfismo.
- Aplicar estos conceptos en sistemas modelables

El Paradigma Procedimental

- **Definición:** En el paradigma procedimental (o imperativo), un programa es una secuencia de comandos. La ejecución de cada comando altera el estado de la memoria.

- **La Limitación Cognitiva:** El programador debe traducir su modelo mental de entidades (ej. 'Vectores', 'Matrices') a una lista de instrucciones secuenciales y direcciones de memoria.

"La programación procedimental se centra en el procesamiento: algoritmos que actúan sobre datos pasivos."



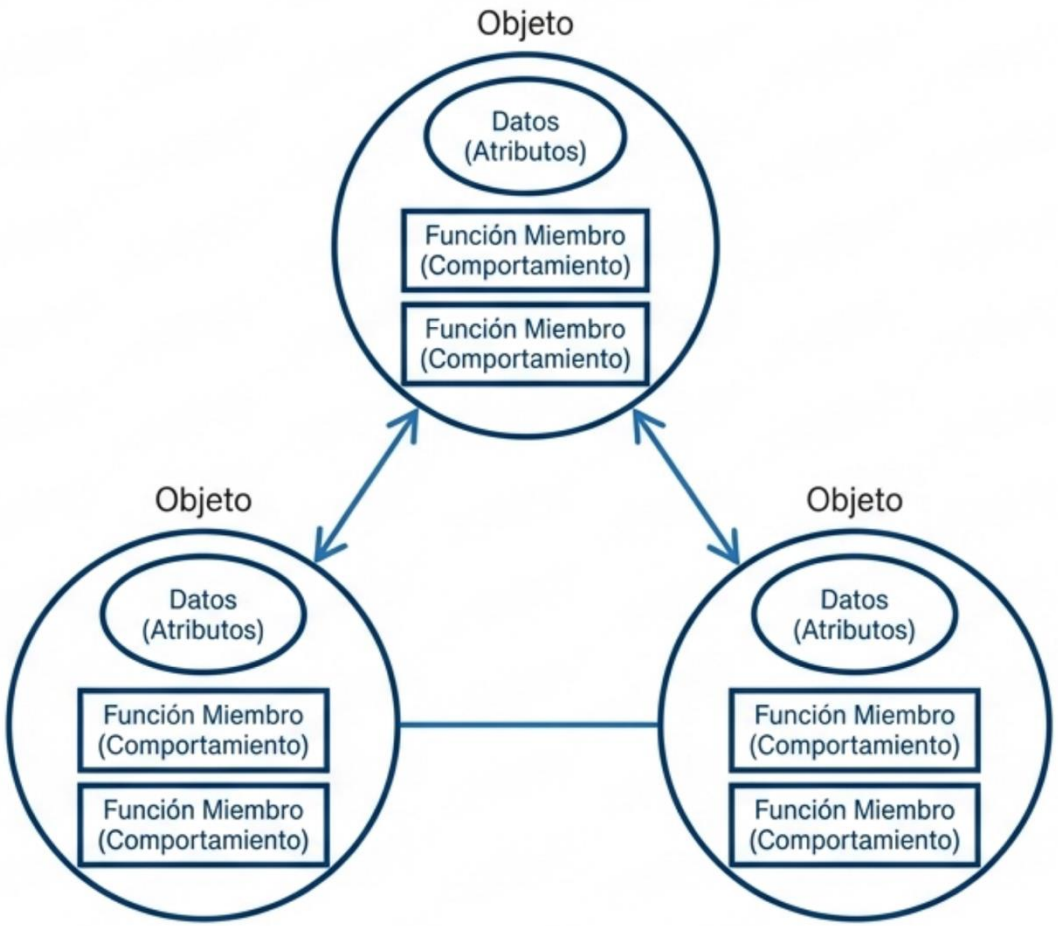
Evolución del estado de memoria en un proceso secuencial.

Programación orientada a Objetos

La POO propone que los datos (*atributos*) y las funciones (*comportamiento*) que los manipulan son inseparables. Esto se alinea con la percepción del mundo real y los objetos matemáticos.

- ***Atributos (Datos):*** Características del estado de una entidad (ej. radio de un círculo, elementos de un conjunto).

- ***Comportamiento (Métodos):*** Operaciones que la entidad puede realizar sobre sí misma o respuestas a estímulos externos.



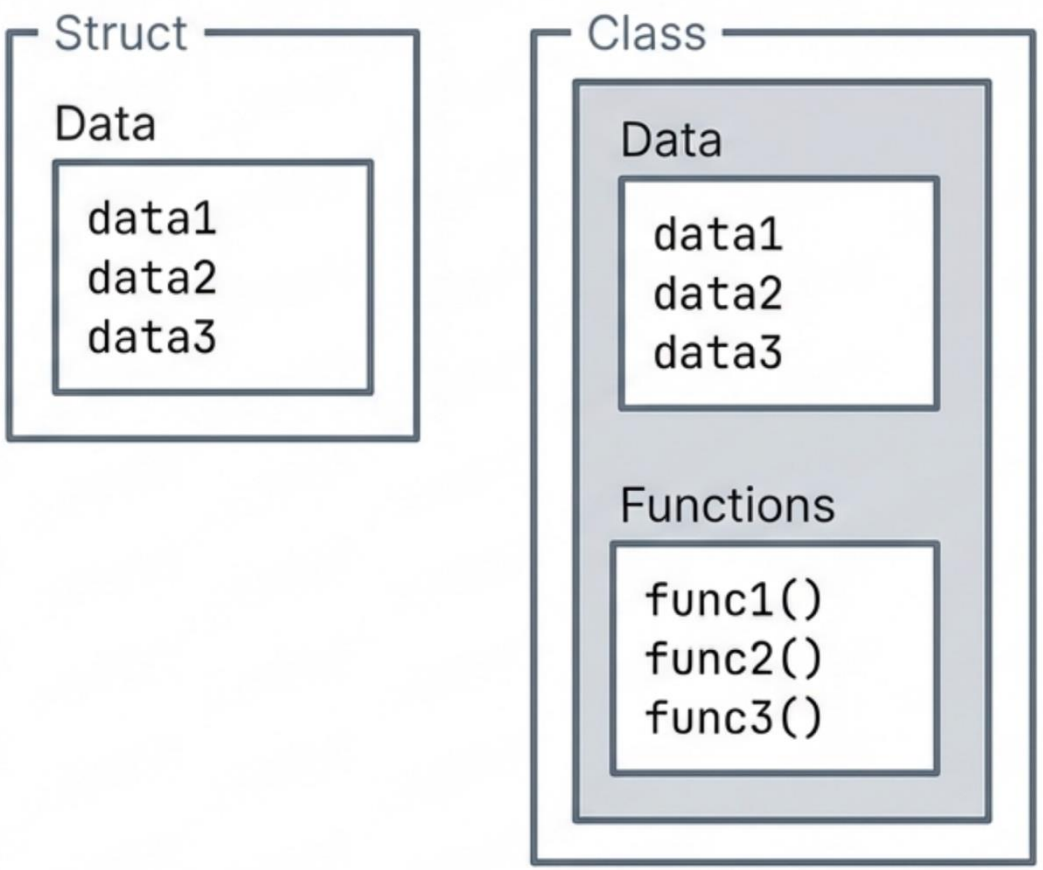
De la estructura al objeto

struct

En C, una estructura es simplemente un contenedor de datos agrupados. Es pasiva; algo externo debe actuar sobre ella.

class

C++ expande la idea de **'struct'** hacia **'class'**. Aquí, los datos y las funciones que los manipulan residen juntos. El objeto protege su propio estado y controla su destino.



Evolucionamos de simples contenedores a entidades inteligentes.

Analogía: Estructuras Algebraicas

En matemáticas, un objeto no se define por cómo se construye, sino por las propiedades que satisface.
La POO permite replicar esta definición axiomática.

Teoría Matemática (Grupo G)

Un conjunto G equipado con una operación binaria tal que:

1. **Clausura:** $\forall\, a, b \in G, a \cdot b \in G$
2. **Asociatividad:** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
3. **Elemento Neutro:** $\exists\, e \in G$ tal que $a \cdot e = a$.
4. **Inverso**: $\forall\, a \in G, \exists\, a^{-1}$ tal que $a \cdot a^{-1} = e$.

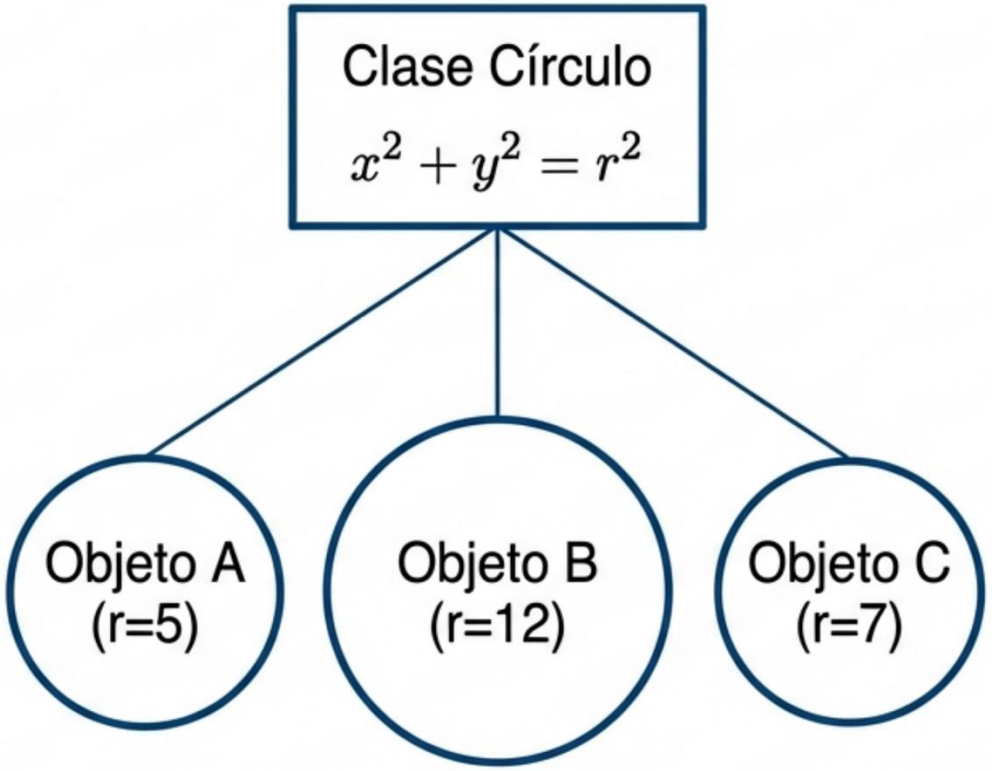
Implementación POO en C++

```
class Grupo {
private:
   Set elementos; // El conjunto G
public:
   // La operación binaria
   Elemento operar(Elemento a, Elemento b) {
     // Implementación que garantiza clausura
   }
   // Invariantes (Axiomas)
   bool verificarAsociatividad();
   Elemento obtenerNeutro();
   Elemento obtenerInverso(Elemento a);
};
```

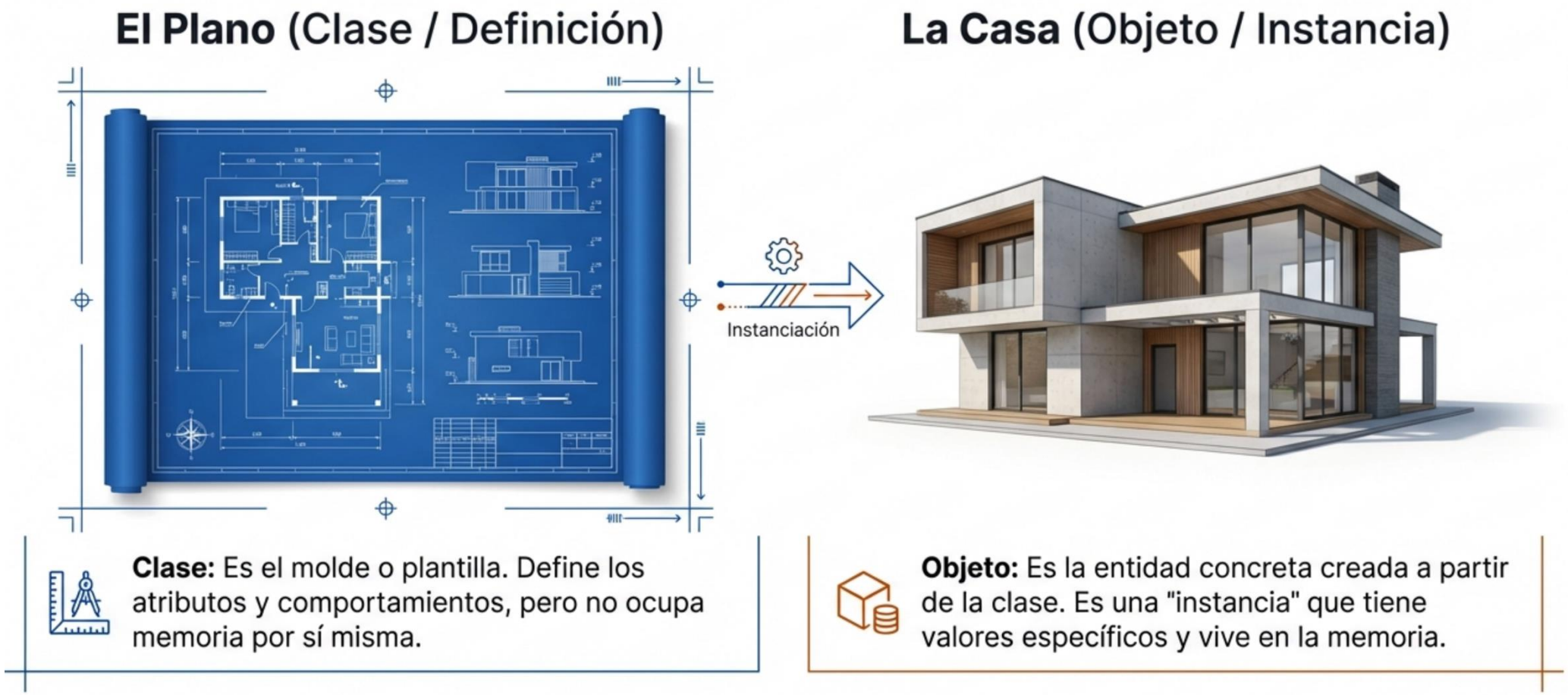
La 'Clase' actúa como la definición formal del tipo algebraico, mientras que los 'Objetos' instanciados son realizaciones específicas de ese grupo.

Clases y Objetos

La Clase (El Tipo): Es el plano o prototipo. En C++, la clase es un Tipo de Dato Definido por el Usuario. Define la estructura y la interfaz pública sin crear datos en memoria. El Objeto (La Instancia): Es la concreción de la clase. Posee identidad única y estado propio en memoria. Al igual que en teoría de conjuntos distinguimos entre el conjunto Z (enteros) y un número específico 5 € Z, en POO distinguimos entre la Clase y el Objeto.



Clase vs. Objeto



Clase en C++

```
class Person {
private:
    long identity;

public:
    void setId(long identity);
    long getId();
};
```

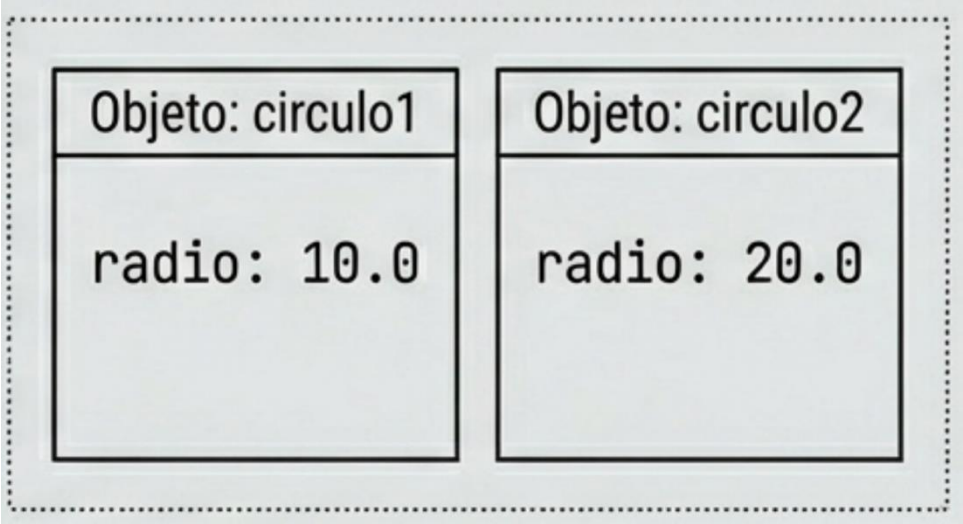
Modificadores de Acceso

Atributos (Datos): Variables que representan las características.

Comportamientos (Funciones): Acciones que el objeto puede realizar.

Objetos: La instancia en la memoria

```
int main() {
  // Instanciación: Creando objetos concretos
  Circulo circulo1;
  Circulo circulo2;

  // Cada objeto tiene su propia copia de 'radio'
  circulo1.setRadio(10.0);
  circulo2.setRadio(20.0);

  return 0;
}
```



La clase es el molde; el objeto es la entidad que ocupa memoria y ejecuta acciones.

El Problema del Acceso Público

Sin protección, la integridad de los datos se rompe fácilmente.

```
class CirculoVulnerable {
public:
 double radio; // ¡Peligro! Acceso público directo
};

int main() {
 CirculoVulnerable c;

 // El compilador lo permite, pero es lógicamente incorrecto
 c.radio = -50.0;

 // Resultado: Área corrupta o cálculo erróneo
 return 0;
}
```

Encapsulamiento: Modificadores de Acceso

```
class Circulo {
private:
  double radio; // Solo accesible desde dentro de la clase

public:
  void setRadio(double r); // Accesible desde todo el programa
};

int main() {
  Circulo c;
  // c.radio = 10.0;  // Error de Compilación: 'radio' es privado
  c.setRadio(10.0);   // Correcto: Acceso controlado
}
```

Conceptos Clave

- private: Datos ocultos (La caja negra).
- public: La interfaz (Los botones externos).

Lógica de Protección: Accesores y Mutadores

El mutador protege el estado del objeto.

El accesor permite leer sin modificar.

```
// Mutador (Setter): Valida los datos antes de asignarlos
void Circulo::setRadio(double valor) {
    if (valor >= 0.0) {
      radio = valor;
    } else {
      // Manejo de error: ignorar o lanzar excepción
      radio = 0.0;
    }
  }

  // Accesor (Getter): Lectura segura (const)
  double Circulo::getRadio() const {
    return radio;
  }
```

Ciclo de Vida: El Constructor

El constructor garantiza que el objeto nunca exista en un estado inválido o no inicializado.

```
class Circulo {
private:
  double radio;
public:
  // Constructor: Mismo nombre que la clase, sin retorno
  Circulo(double r) {
    if (r >= 0) radio = r;
    else radio = 0;
  }
};

int main() {
  // El objeto nace ya inicializado y válido
  Circulo c1(5.5);
  return 0;
}
```

Ciclo de Vida: El Destructor

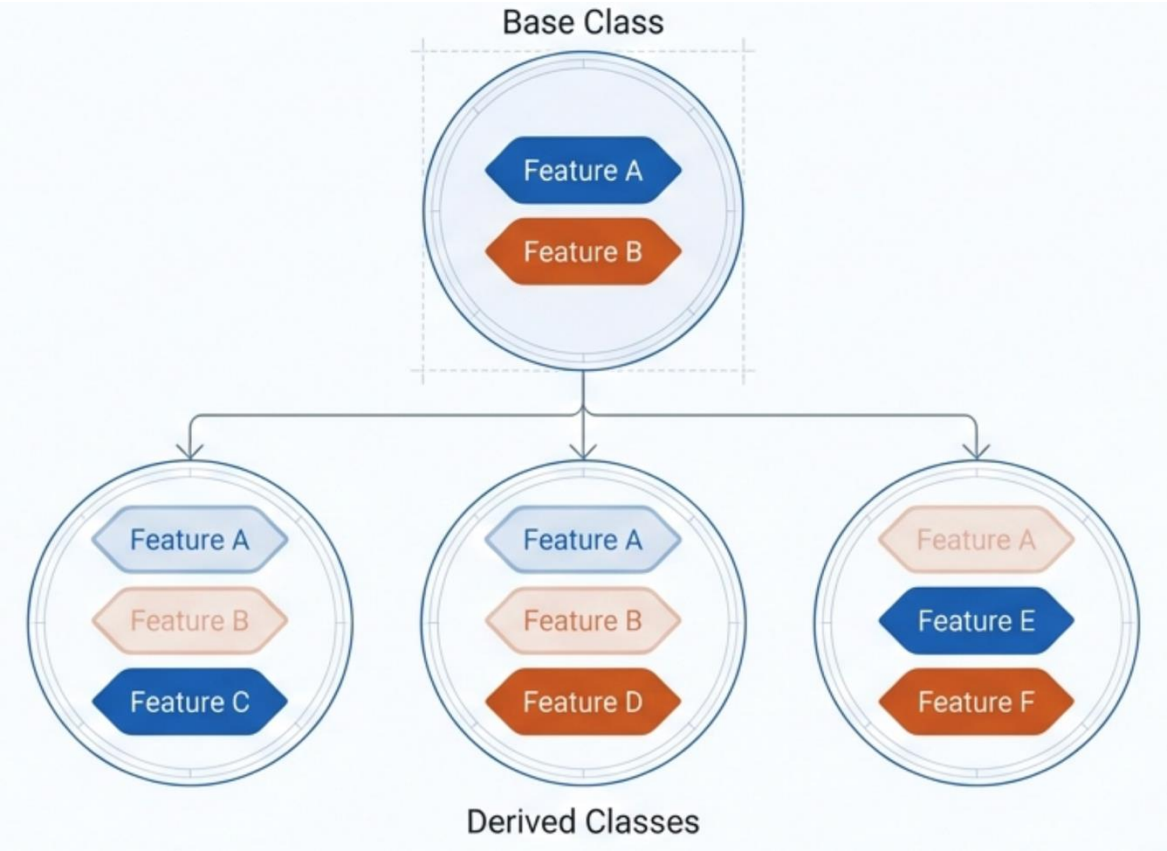
```
class Circulo {
public:
    // Constructor
    Circulo() {
      // Inicialización de recursos
    }

    // Destructor: Prefijo tilde (~), limpieza automática
    ~Circulo() {
        // Liberación de recursos al salir de ámbito
        // cout << "Objeto destruido";
    }
};
```

Se invoca automáticamente al finalizar el bloque scope

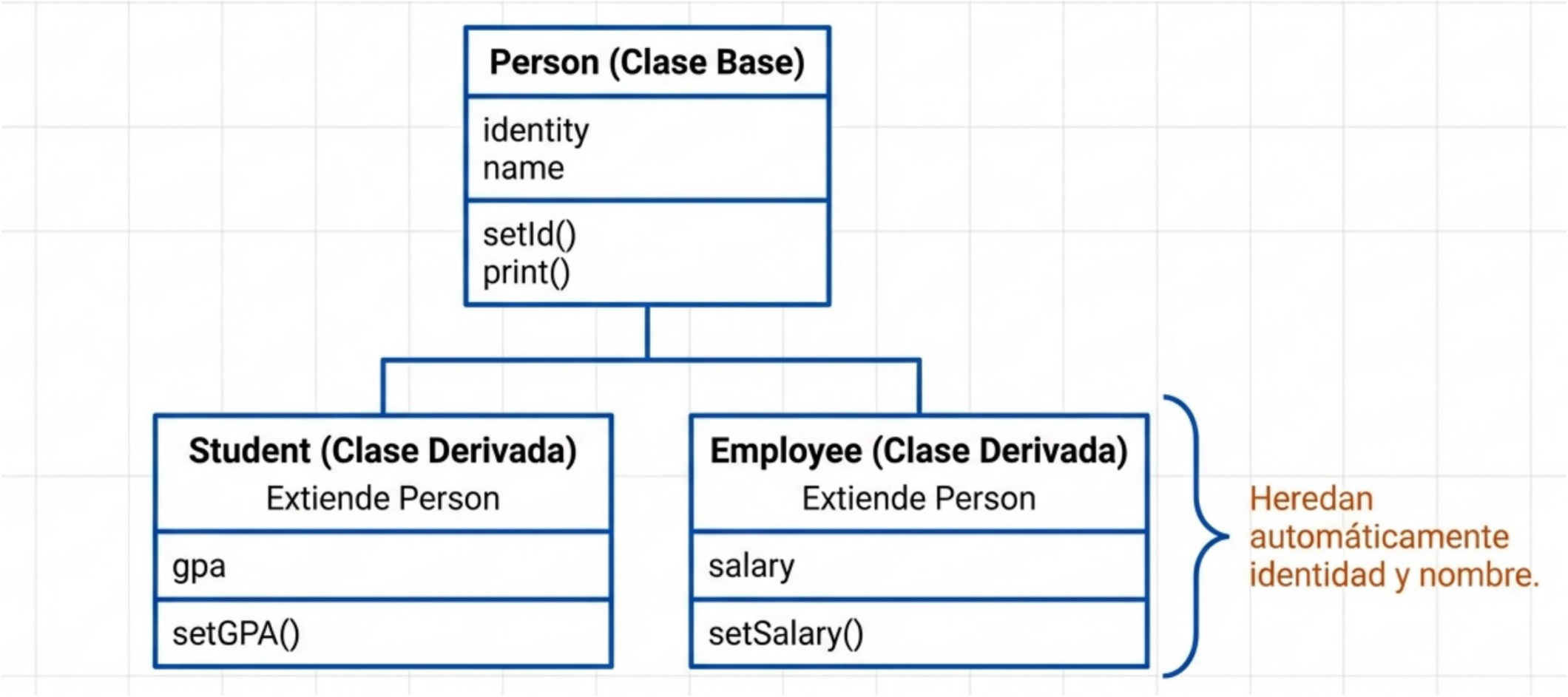
Herencia

La herencia nos permite crear nuevas clases basadas en clases existentes.

Relación 'Es-un' (Is-A):
- Un Estudiante **es una** Persona.
- Un Caballo **es un** Animal.



Herencia


Person (Clase Base)
identity
name
setId()
print()
Student (Clase Derivada)
Extiende Person
gpa
setGPA()
Employee (Clase Derivada)
Extiende Person
salary
setSalary()
Heredan automáticamente identidad y nombre.

Herencia: Sintaxis

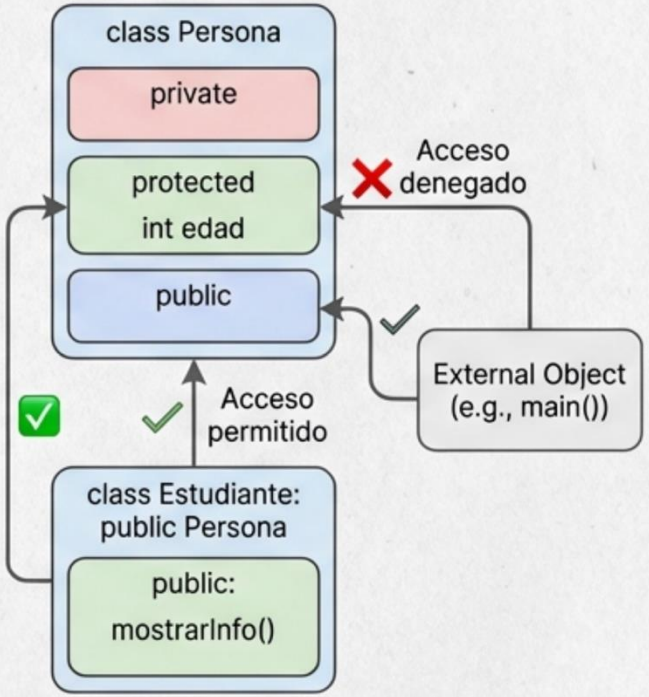
```
// Clase Base
class Person {
public:
  void setId(long identity);
  long getId();
};
```

```
// Clase Derivada (Student hereda de Person)
class Student : public Person {
private:
  double gpa; // Atributo nuevo
public:
  void setGPA(double gpa);
};
```

Esta sintaxis absorbe las capacidades públicas de Person.

El Modificador Protected

```cpp
class Persona {
protected:
    // Visible para 'Persona' y sus herederos ('Estudiante')
    // Invisible para 'main()'
    int edad;
};

class Estudiante : public Persona {
public:
    void mostrarInfo() {
        // Acceso directo permitido gracias a 'protected'
        cout << "Edad: " << edad;
    }
};
```



Protected ofrece privacidad para el exterior, pero confianza para la jerarquía de herencia.

La POO como Modelo de Abstracción

1. Modularidad: "Divide y Vencerás" en Memoria

- Hemos transformado coordenadas abstractas en un objeto `Punto` y una trayectoria física en un objeto `Camino`.
- Dividimos el problema en piezas independientes que interactúan entre sí. Un cambio en cómo calculamos la distancia en `Punto` no afecta la lógica de cómo `Camino` suma esas distancias.

2. Mantenibilidad y Encapsulamiento

- El uso de modificadores `private` y `protected` crea una "caja negra".
- Al proteger los datos y usar **referencias constantes** (`const &`), garantizamos que el estado interno del objeto no sea corrompido. Podemos optimizar el código interno de la clase sin que el usuario final (el `main`) tenga que cambiar una sola línea.

3. Flexibilidad y Herencia

- La relación "es un" (*is-a*).
- Un `CaminoFisico` hereda automáticamente toda la gestión de memoria dinámica y lógica del `Camino` base. Esto permite extensibilidad; hoy calculamos energía, mañana podríamos heredar para calcular tiempos de llegada o fricción, reutilizando siempre el código base.

4. Contrato de Interfaz y Ciclo de Vida

- La clase define el "qué", no el "cómo".
- El **Constructor** y **Destructor** son contratos de gestión de recursos. Hemos aprendido que en C++ la libertad de usar el **Heap** (`new[]`) conlleva la responsabilidad de liberar la memoria (`delete[]`). El objeto es un ente vivo con un inicio y un fin definido.